

Reviewer Pack — Minimal Geometric Entropy of Metric Trees and Resolution Pro...

Entry 7a67672e093c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:44:25 UTC

Minimal Geometric Entropy of Metric Trees and Resolution Proof Width Inequality

Entry ID: 7a67672e093c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 02:44:25 UTC

1. Conjecture

Field A (mathematical branch): Metric Geometry (Metric Trees) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

For every Boolean satisfiability instance φ with n variables, the minimal geometric entropy ($H_{\min}$) of its associated metric tree $T(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $H_{\min}(T(\varphi)) = \Omega(w(\varphi))$.

Rationale (proposer's reasoning):

Metric trees provide a discrete analog to Riemannian manifolds, and their geometric entropies are known to capture certain properties of the underlying space. By examining how the entropy changes with the resolution proof width, we might uncover connections between geometric properties and computational complexity, potentially revealing new insights into the nature of NP-hardness.

Taxonomy category: Metric_Geometry_x_Complexity_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5237a244831dd4bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each Boolean satisfiability instance φ with n variables ($n \leq 40$), if the minimal geometric entropy $H_{\min}(T(\varphi))$ is greater than or equal to half of the resolution proof width $w(\varphi)$ for all seeds and the mean difference between $H_{\min}$ and $\Omega(w)$ across seeds is less than or equal to a predefined constant k .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "metric geometry" AND "resolution proof complexity" AND "geometric entropy" - "metric tree" AND "resolution proof width" AND "Boolean satisfiability" - "minimal geometric entropy" AND "resolution proof width" AND "metric trees"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clauses(n):
        return [[random.randint(0, n-1) for _ in range(random.randint(2, 4))] for _ in range(random.randint(2, 4))]

    def construct_metric_tree(clauses):
        n = len(clauses)
        tree = {}
        for i in range(n):
            tree[i] = {}
        for clause in clauses:
            for i in clause:
                for j in clause:
                    if i != j and (i not in tree[j] or tree[j][i] > 1):
                        tree[i][j] = 1
                        tree[j][i] = 1
        return tree

    def calculate_geometric_entropy(tree, n):
        total_edges = sum(sum(1 for _ in values) for values in tree.values()) // 2
        entropy = 0
        for i in range(n):
            degree = len(tree[i])
            if degree > 0:
                entropy += math.log(degree)
        return -entropy / n

    def calculate_resolution_width(clauses):
        # Simplified DPLL solver to estimate resolution width
```

```

def dpll(clauses, assignment):
    if not clauses:
        return True
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        if dpll(new_clauses, assignment + {literal: True}):
            return True
        if dpll(new_clauses, assignment + {-literal: True}):
            return True
    else:
        literals = set(lit for clause in clauses for lit in clause)
        literal = next(lit for lit in literals if all(lit not in c and -lit not in c for c in clauses))
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        if dpll(new_clauses, assignment + {literal: True}):
            return True
    return False

width = 0
for literal in range(-100, 101): # Simplified search space
    clauses_copy = [c[:] for c in clauses]
    if dpll(clauses_copy, {}):
        width += 1
return width

n = random.randint(5, 40)
clauses = generate_clauses(n)
tree = construct_metric_tree(clauses)
H_min = calculate_geometric_entropy(tree, n)
w = calculate_resolution_width(clauses)

if H_min < 0 or w <= 0:
    return {
        "metric_name": "H_min -  $\Omega(w)$ ",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "negative_values"
    }

metric_value = abs(H_min - (0.5 * w))
conjecture_holds = H_min >= 0.5 * w
counterexample = "" if conjecture_holds else f"H_min={H_min},  $\Omega(w)=0.5*\{w\}$ "

return {
    "metric_name": "H_min -  $\Omega(w)$ ",
    "metric_value": metric_value,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_675dfd89.py", line 109, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_675dfd89.py", line 76, in run_trial
    tree = construct_metric_tree(clauses)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_675dfd89.py", line 32, in construct_metric_tree
    if i != j and (i not in tree[j] or tree[j][i] > 1):
                   ~~~~~^
KeyError: 32

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be verified. | next: Investigate and fix the crash in the test code to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12769
2	preregistration	ollama_remote	glm4:latest	0	0	10020
3	novelty	ollama_remote	glm4:latest	0	0	16450
4	novelty	ollama_remote	glm4:latest	0	0	8861
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14588
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11090
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13776
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13929
9	judge	ollama_remote	glm4:latest	0	0	30452

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131935 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7a67672e093c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7a67672e093c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7a67672e093c.tar.gz` (if generated)